167 XORs for Rank-47 4×4 Matrix Multiplication in Characteristic Two

Greg Sidebottom

Draft — October 17, 2026

Abstract

Rank-47 bilinear schemes multiply 4×4 matrices with 47 multiplications over any ring of characteristic two; at least three inequivalent ones are publicly available. Applied recursively, a scheme whose three linear maps use q additions (XORs) costs $(1 + q/31)\, n^{\log_4 47} + o(n^{\log_4 47})$ operations, so q fixes the leading constant. The best count known for a rank-47 scheme was 170 (Dumas, Pernet, Sedoglavic, Stapleton and Tichavský, 2026). We give a rank-47 scheme over $\mathbb{F}_2$ whose maps cost $44+49+74 = 167$ XORs, leading constant $6+12/31$, and 168 for the published scheme itself. The new scheme lies in the symmetry orbit of Kauers and Moosbauer's rank-47 scheme. It was found by simulated annealing over the orbit, scored by a Boyar–Peralta heuristic with exact distances, followed by an exact search for the minimum number of non-target XORs on each side; for this scheme all three counts are proved minimal. Across the three known classes, every annealing chain ends at 168 under the heuristic, and only the Kauers–Moosbauer class reaches 167 after exact refinement. All programs are checked by an independent verifier and by PLinOpt's `SLPchecker`. We also summarize the current rank and addition records for 4×4 matrix multiplication with and without change of basis.

1 Introduction

Strassen's algorithm applied to 2×2 blocks of 2×2 matrices multiplies 4×4 matrices with 49 multiplications over any ring [28]. AlphaTensor lowered this to 47 in characteristic two [12]; Kauers and Moosbauer found an inequivalent rank-47 scheme with flip graphs [16]; and a third, attributed in [10] to AlphaEvolve [21], is the one whose programs PLinOpt distributes [11]. Outside characteristic two the record is 48, over the complex numbers by AlphaEvolve and over any ring containing $1/2$ by Dumas, Pernet and Sedoglavic [8].

Used recursively, a scheme's multiplication count fixes the exponent and its additions fix the constant. For an $\langle n_0 \times n_0 \times n_0 : r \rangle$ scheme whose linear maps use q additions and scalings in total,

$$F(n) = r\, F(n/n_0) + q\,(n/n_0)^2 \quad \Longrightarrow \quad F(n) = \Big(1 + \frac{q}{r - n_0^2}\Big) n^{\log_{n_0} r} + o\big(n^{\log_{n_0} r}\big),$$

so at $r = 47$, $n_0 = 4$ the constant is $1 + q/31$. Over $\mathbb{F}_2$ every addition is an XOR and no scalings occur. Recent work drove the rank-48 count from 315 operations [8] down to 204 [10]; for rank 47, the same paper reports the best known program at 170 XORs, constant $6 + 15/31$.

Contributions.

- A rank-47 scheme over $\mathbb{F}_2$ with $167 = 44 + 49 + 74$ XORs (constant $6 + 12/31 \approx 6.387$), given in full in Appendix A, and 168 for the published scheme (Section 5). For the new scheme each side's count is proved minimal.

- An exact method for the minimum XOR count of a side with few non-target signals (Section 4.3): the problem reduces to choosing those signals, and iterative deepening over their number is exact and fast up to four of them.
- A search over each scheme's symmetry orbit by simulated annealing, and the observation that all three known classes plateau at 168 under heuristic scoring (Section 5).
- A summary of the current 4×4 records (Section 2).

2 Records for 4×4 matrix multiplication

Table 1 lists the best known ranks; Tables 2 and 3 the best known addition counts, without and with a change of basis. The leading constants compare only within one rank, since each rank recurses with its own exponent.

Table 1: Best known rank of $\langle 4, 4, 4 \rangle$.

setting	rank	source
characteristic 2 (e.g. $\mathbb{F}_2$)	47	AlphaTensor [12]; inequivalent schemes by Kauers–Moosbauer [16] and AlphaEvolve [21, 10]
rings containing 1/2	48	AlphaEvolve (over $\mathbb{C}$) [21]; rational coefficients [8]
all rings	49	Strassen applied twice [28]

Table 2: Best known additions (with scalings) without change of basis; constant of $n^{\log_4 r}$.

rank (valid over)	exponent	ops	constant	source
47 (char. 2)	$\log_4 47 \approx 2.777$	170	$6 + 15/31$	[11], cited in [10]
		167	$\mathbf{6 + 12/31}$	this paper
48 (rings with 1/2)	$\log_4 48 \approx 2.792$	315	$10 + 27/32$	[8]
		204	$7 + 3/8$	[10] [a]
49 (all rings)	$\log_2 7 \approx 2.807$	165	6	[31], applied twice
		156	$5 + 8/11$	[11], cited in [10]

[a] The PLinOpt repository also holds a 202-operation variant (data file `4x4x4_48_202`, contributed by R. Patel, October 2026).

Table 3: Best known constants with a change of basis (conversion costs are lower order). d is the encoding dimension; $d = 16$ is a square basis change.

rank	basis change	recursive ops	constant	source
49	square (2×2, twice)	12 per 2×2 step	5	[14]
48	square ($d = 16$)	173	$6 + 13/32$	[10]
48	rectangular ($d = 24$)	122	$6 + 1/12$	[10]
48	rect. ($d = 24$: A, B; 32: C)	$32 + 32 + 32$	$5 + 2/3$	[10]
47	—	—	—	none published

Two remarks. First, the rectangular encodings of Beniamini and Schwartz [2, 1] are a different cost model from Karstadt and Schwartz's square basis change [14]: with them a 3×3 rank-23 base already reaches constant 2 [2], the constant of the classical algorithm. Second, the 3×3 analogue of this paper's question has seen a run of improvements: 60 [27], 59 [17], 58 [22], 56 [29] and 55 additions [24, 15] without change of basis, and 52 [20] then 51 additions per step [26, 25] with a square basis change. Mårtensson and Stankovski Wagner [18, 19] and Perminov [23] study addition reduction for many formats; Dumas, Pernet and Sedoglavic [8, 9, 10] and the PLinOpt tools [11] are the reference for 4×4.

3 Model, lower bounds and symmetry

A rank-r scheme for $\langle 4, 4, 4 \rangle$ over $\mathbb{F}_2$ is a triple of matrices $L \in \mathbb{F}_2^{r \times 16}$, $R \in \mathbb{F}_2^{r \times 16}$, $P \in \mathbb{F}_2^{16 \times r}$ with

$$\operatorname{vec}(AB) = P\big((L \operatorname{vec} A) \odot (R \operatorname{vec} B)\big)$$

for all A, B, where vec lists entries row by row and $\odot$ is the entrywise product. Equivalently, product k is $m_k = \langle U_k, A\rangle\langle V_k, B\rangle$ for 4×4 matrices U_k, V_k (rows k of L and R) and $AB = \sum_k m_k W_k$ (W_k is column k of P). Since the identity holds over the field $\mathbb{F}_2$, it holds for entries in any ring of characteristic two, commutative or not, so the scheme recurses on blocks.

Cost. For a matrix M with rows over the inputs, let $\mathrm{X}(M)$ be the fewest XOR gates of a straight-line program computing all rows of M from the inputs (copies are free). The cost of a scheme is $\mathrm{X}(L) + \mathrm{X}(R) + \mathrm{X}(P)$. The output side is handled through its transpose: by the transposition principle [6] a linear program for a matrix without zero rows or columns can be reversed into one for its transpose, and $\mathrm{X}(P) = \mathrm{X}(P^{\mathsf{T}}) + (r - 16)$. All three sides are therefore instances of one problem: compute a set of r vectors of $\mathbb{F}_2^{16}$ from the 16 unit vectors. Over $\mathbb{F}_2$ the shortest such program is NP-hard to find in general [3].

Floor. Every target that is not a unit vector needs a gate of its own, so $\mathrm{X}(M) \geq |T(M)|$, where $T(M)$ is the set of distinct non-unit rows. For a scheme this gives the floor $|T(L)| + |T(R)| + |T(P^{\mathsf{T}})| + 31$.

Symmetry. If $(U_k, V_k, W_k)_k$ is a scheme, so is $(X^{\mathsf{T}} U_k Y^{-\mathsf{T}},\ Y^{\mathsf{T}} V_k Z^{-\mathsf{T}},\ X^{-1} W_k Z)_k$ for invertible X, Y, Z (substitute $A \mapsto XAY^{-1}$, $B \mapsto YBZ^{-1}$), and so are the images under the cyclic symmetry $(U, V, W) \mapsto (V, W^{\mathsf{T}}, U^{\mathsf{T}})$ and the transposition $(U, V, W) \mapsto (V^{\mathsf{T}}, U^{\mathsf{T}}, W^{\mathsf{T}})$ of the matrix multiplication tensor [7]. Over $\mathbb{F}_2$ the orbit of a scheme has up to $6 \cdot 20160^3 \approx 4.9 \cdot 10^{13}$ points, all with the same multiplications but different linear maps, and hence different XOR counts. The multiset of rank triples $(\operatorname{rank} U_k, \operatorname{rank} V_k, \operatorname{rank} W_k)$, taken up to a common permutation of the three coordinates, is constant on an orbit; it separates the three published rank-47 schemes, which are therefore pairwise inequivalent. None of the three has two products sharing a factor, so each is an isolated vertex of the flip graph [16] and flip walks cannot leave it.

4 Method

4.1 Boyar–Peralta with exact distances

Boyar and Peralta's heuristic [5, 4] keeps a base S of computed signals, initially the unit vectors, and the distance $\delta(t)$ of each target: one less than the fewest elements of S summing to t. Each

step adds a target at distance one if there is one, and otherwise the sum of two base elements that minimizes the total distance, ties broken by the larger sum of squared distances. With 16 inputs the distances are exact and cheap: a table of the 2^{16} values $k(v)$ (fewest base elements summing to v) starts at the Hamming weights and is updated in one pass per new signal s by $k(v) \leftarrow \min(k(v), 1 + k(v + s))$. We add random tie breaking and optional slack, in the spirit of Tan and Peyrin [30], run many restarts, delete unused gates, and reverse the program for P^{T} into one for P. A pilot variant (complete greedily after each of the best few candidates and commit to the best completion) never improved on the plain heuristic in our instances.

4.2 Searching the symmetry orbit

A point of the orbit is a slot symmetry together with $(X, Y, Z) \in GL_4(\mathbb{F}_2)^3$; X affects the L and P sides, Y the L and R sides, Z the R and P sides. We tried coordinate descent (each step scans all 20160 values of one coordinate, shortlists the best by floor and weight, and scores the shortlist with the heuristic) and simulated annealing whose moves multiply one coordinate by an elementary matrix $I + E_{ij}$ on either side and rescore the two affected sides. Annealing was clearly better: on the AlphaTensor class two chains of 3000 moves reached 168, where 240 coordinate-descent restarts reached 169 once. Every orbit point that is written out is checked against all 256 basis products.

4.3 Exact minimization with few helper signals

Call the gates of a program that compute targets *target gates* and the others *helpers*. A program for T has $|T| + h$ gates when it uses h helpers. The following lemma removes all choices except the helpers. For $S \subseteq \mathbb{F}_2^{16}$ let $\mathrm{cl}(S)$ be the least superset of S that contains every target equal to the sum of two of its elements.

Lemma 4.1. *A program computing T with exactly h helpers exists if and only if there are $x_1, \ldots, x_h$ such that, with $S_0 = \mathrm{cl}(\{e_1, \ldots, e_{16}\})$ and $S_i = \mathrm{cl}(S_{i-1} \cup \{x_i\})$, each x_i is a sum of two elements of S_{i-1} and $T \subseteq S_h$.*

Proof. Given such x_i, build the program by adding targets as soon as they are sums of two available signals and the x_i in order: every gate is a target or one of the h helpers. Conversely, take a program with h helpers, listed in program order as $x_1, \ldots, x_h$. By induction on the position in the program, every signal before x_i lies in S_{i-1}: a target gate is the sum of two earlier signals, which lie in the current closure, so the closure contains it; hence each x_i is a sum of two elements of S_{i-1}, and finally $T \subseteq S_h$. □

Two observations prune the search over helper sequences without losing solutions. The last helper must make some missing target a sum of two available signals, so $x_h \in (T \setminus S_{h-1}) + S_{h-1}$. And if x_{i+1} is already a sum of two elements of S_{i-1}, the two helpers can be exchanged without changing S_{i+1}; bubble-sorting any solution therefore yields one in which such adjacent pairs are increasing (as integers), and only those are explored. Iterative deepening over h returns the exact minimum. Candidates are tried in order of how many targets they make immediately available. With 16 inputs and 41–45 targets, refuting $h \leq 3$ takes milliseconds to seconds and finding or refuting $h = 4$ takes seconds to a minute; each further level costs roughly a hundred times more.

4.4 Pipeline

Annealing chains record every distinct point whose heuristic total is at most 169 (about 14,500 points over the three classes); each point is then refined side by side, searching exactly for a

program one helper shorter than the heuristic's within a time limit. Side results are cached by target set, since neighbouring points share two of their three sides.

5 Results

The published scheme. PLinOpt's programs for its rank-47 scheme cost $44 + 44 + 82 = 170$. The heuristic alone gives $44+44+80 = 168$ for the same scheme: P^{T} needs 49 XORs, not 51. Here the floor is low ($37 + 37 + 34 + 31 = 139$) but each side carries many helpers. The exact search rules out five or fewer helpers on L, so $\mathrm{X}(L) \geq 43$ against the heuristic's 44; deeper levels were out of reach.

The orbit plateau. Table 4 lists the best orbit points. Under heuristic scoring every class stops at 168: 24 of 24 annealing chains for the AlphaTensor and published classes, 23 of 24 for the Kauers–Moosbauer class. At an AlphaTensor-class point with sides $44 + 48 + 45$ (the last counted for P^{T}) the exact search proves all three sides optimal, so that scheme costs exactly 168.

Table 4: Best orbit points per class (total XORs, sides $L + R + P$).

class	as given	heuristic, best point	exact refinement
published [11] (AlphaEvolve)	170	$168 = 44 + 44 + 80$	not refined
AlphaTensor [12]	179	$168 = 44 + 48 + 76$	168 (0 of 1203 below)
Kauers–Moosbauer [16]	186	$168 = 44 + 50 + 74$	$\mathbf{167 = 44 + 49 + 74}$

The 167 scheme. Exact refinement lowers the most expensive side of many Kauers–Moosbauer-class points by one: of the first 537 points refined, 211 reach 167, 234 stay at 168 and 92 at 169; none reaches 166. The scheme of Appendix A is one of them. It is the image of Kauers and Moosbauer's scheme under the slot symmetry $(U, V, W) \mapsto (W, V^{\mathsf{T}}, U)$ followed by the substitution with (X, Y, Z) given by `54a8`, `59f8`, `a512` in Appendix A's encoding (recomputing this image reproduces the appendix exactly). Its sides have 41, 45 and 41 distinct non-unit targets (floor 158), and the exact search gives

$$\mathrm{X}(L) = 44 \text{ (3 helpers)}, \qquad \mathrm{X}(R) = 49 \text{ (4 helpers)}, \qquad \mathrm{X}(P^{\mathsf{T}}) = 43 \text{ (2 helpers)},$$

each with the next smaller helper count refuted, so $\mathrm{X}(P) = 43 + 31 = 74$ and 167 is the exact minimum for this scheme without change of basis. The heuristic finds 50 for R; the exact search finds the four-helper program.

Leading constant. The scheme gives $F(n) = \frac{198}{31}\, n^{\log_4 47} + o(n^{\log_4 47})$ ring operations (additions and multiplications) over rings of characteristic two, against $\frac{201}{31}$ before.

6 Verification and artifacts

The scheme and programs are in the repository https://github.com/gsidebottom/logic, directory `matmul/x47/` (`best/x167/`: `x167_{L,R,P}.sms` and `.slp` in PLinOpt formats). Two independent checks pass:

- `x47check.py` (written for this purpose, independent of the search code) evaluates each program over $\mathbb{F}_2$, compares every output with the corresponding matrix row, counts XORs as the number of terms minus one per line, and checks the scheme on all 256 pairs of basis matrices, which by bilinearity proves $P((L\,\mathrm{vec}\,A) \odot (R\,\mathrm{vec}\,B)) = \mathrm{vec}(AB)$; it reports `OK rank 47: L 44 + R 49 + P 74 = 167 XORs (Brent 256/256 basis pairs)`.
- PLinOpt's `SLPchecker -q 2 -M x167_S.sms < x167_S.slp` reports `<44, 0>`, `<49, 0>` and `<74, 0>` for $S = L, R, P$.

The search tool (`src/bin/xor47.rs`: `bp`, `scheme`, `orbit`, `exact`, `refine`) regenerates every number in Section 5.

7 Outlook

The three known classes all stop at 168 under the heuristic and the best refined point is 167, so further progress most likely needs new rank-47 classes. All known classes are flip-isolated, which rules out plain flip walks. SAT-based search found thousands of 3×3 schemes [13]; we are running a SAT search for 4×4 schemes invariant under the cyclic symmetry, which divides the unknowns by three and made the analogous 3×3 search tractable. Other directions are an exact search reaching five or six helpers (it would settle, for example, whether the published scheme's L side admits 43), the first alternative-basis constants for rank 47, and rectangular encodings in the sense of [2].

Acknowledgments

This work was carried out in an extended interactive collaboration with Claude (Anthropic), which implemented the search and verification tools and much of this text under the author's direction and review. Every claim is checked mechanically by the tools above; the results rest on those checks rather than on trust in either the author or the tools.

References

[1] G. Beniamini, N. Cheng, O. Holtz, E. Karstadt, O. Schwartz, *Sparsifying the operators of fast matrix multiplication algorithms*, arXiv:2008.03759 (2020).

[2] G. Beniamini, O. Schwartz, *Fast matrix multiplication via sparse decomposition*, Proc. SPAA 2019, 11–22.

[3] J. Boyar, P. Matthews, R. Peralta, *On the shortest linear straight-line program for computing linear forms*, Proc. MFCS 2008, LNCS 5162, 168–179.

[4] J. Boyar, P. Matthews, R. Peralta, *Logic minimization techniques with applications to cryptology*, J. Cryptology 26 (2013) 280–312.

[5] J. Boyar, R. Peralta, *A new combinational logic minimization technique with applications to cryptology*, Proc. SEA 2010, LNCS 6049, 178–189.

[6] P. Bürgisser, M. Clausen, M. A. Shokrollahi, *Algebraic Complexity Theory*, Springer, 1997.

[7] H. F. de Groote, *On varieties of optimal algorithms for the computation of bilinear mappings I. The isotropy group of a bilinear mapping*, Theoret. Comput. Sci. 7 (1978) 1–24.

[8] J.-G. Dumas, C. Pernet, A. Sedoglavic, *A non-commutative algorithm for multiplying* 4×4 *matrices using* 48 *non-complex multiplications*, arXiv:2506.13242 (2025).

[9] J.-G. Dumas, C. Pernet, A. Sedoglavic, *A more accurate rational non-commutative algorithm for multiplying* 4×4 *matrices using* 48 *multiplications*, arXiv:2603.18699 (2026).

[10] J.-G. Dumas, C. Pernet, A. Sedoglavic, J. Stapleton, P. Tichavský, *Fast matrix multiplication via recursive* $\langle 4 \times 4 \times 4 : 48 \rangle$ *algorithms into practice*, arXiv:2609.12027 (2026).

[11] J.-G. Dumas, B. Grenet, C. Pernet, A. Sedoglavic, *PLinOpt, a collection of C++ routines handling linear & bilinear programs*, https://github.com/jgdumas/plinopt (data file `4x4x4_47_mod_2`, September 2026).

[12] A. Fawzi et al., *Discovering faster matrix multiplication algorithms with reinforcement learning*, Nature 610 (2022) 47–53.

[13] M. J. H. Heule, M. Kauers, M. Seidl, *Local search for fast matrix multiplication*, Proc. SAT 2019, LNCS 11628, 155–163.

[14] E. Karstadt, O. Schwartz, *Matrix multiplication, a little faster*, Proc. SPAA 2017, 101–110; J. ACM 67 (2020).

[15] S. Karunaratne, A. Idamekorala, *55 additions suffice for* 3×3 *matrix multiplication at rank 23*, arXiv:2607.28676 (2026).

[16] M. Kauers, J. Moosbauer, *Flip graphs for matrix multiplication*, Proc. ISSAC 2023; arXiv:2212.01175.

[17] E. Mårtensson, P. Stankovski Wagner, J. Stapleton, *A rank 23 algorithm for multiplying* 3×3 *matrices with an arithmetic complexity of 59*, arXiv:2601.05272 (2026).

[18] E. Mårtensson, P. Stankovski Wagner, *The number of the beast: reducing additions in fast matrix multiplication algorithms for dimensions up to 666*, IACR ePrint 2024/2063.

[19] E. Mårtensson, P. Stankovski Wagner, *On why and how to minimize the arithmetic complexity of fast matrix multiplication algorithms*, IACR ePrint 2026/849.

[20] H. Nielsen, *A 52-addition, rank-23 scheme for exact* 3×3 *matrix multiplication*, arXiv:2609.06588 (2026).

[21] A. Novikov et al., *AlphaEvolve: a coding agent for scientific and algorithmic discovery*, arXiv:2506.13131 (2025).

[22] A. I. Perminov, *A 58-addition, rank-23 scheme for general* 3×3 *matrix multiplication*, arXiv:2512.21980 (2025).

[23] A. I. Perminov, *Parallel heuristic exploration for additive complexity reduction in fast matrix multiplication*, arXiv:2512.13365 (2025).

[24] G. Sidebottom, *A 55-addition rank-23 scheme for* 3×3 *matrix multiplication via exact two-sided minimization*, Zenodo, doi:10.5281/zenodo.21240904 (July 2026).

[25] G. Sidebottom, *51 additions per recursive step for rank-23* 3×3 *matrix multiplication, machine-checked over arbitrary rings*, Zenodo, doi:10.5281/zenodo.23305910 (October 2026).

[26] J. Stapleton, A. Perminov, *A 51-addition alternative-basis kernel for rank-23* 3×3 *matrix multiplication*, arXiv:2609.32388 (2026).

[27] J. Stapleton, *A 60-addition, rank-23 scheme for exact* 3×3 *matrix multiplication*, arXiv:2508.03857 (2025).

[28] V. Strassen, *Gaussian elimination is not optimal*, Numer. Math. 13 (1969) 354–356.

[29] Y. Sun, *An exact 56-addition, rank-23 scheme for general* 3×3 *matrix multiplication*, arXiv:2604.27645 (2026).

[30] Q. Q. Tan, T. Peyrin, *Improved heuristics for short linear programs*, IACR Trans. Cryptogr. Hardw. Embed. Syst. 2020(1) 203–230.

[31] S. Winograd, *On multiplication of* 2×2 *matrices*, Linear Algebra Appl. 4 (1971) 381–388.

A The 167-XOR scheme

A.1 The scheme

Product k is $m_k = \langle U_k, A\rangle \langle V_k, B\rangle$ and $C = \sum_k m_k W_k$ over $\mathbb{F}_2$. Each 4×4 coefficient matrix is written as four hexadecimal digits, digit i ($i = 0, \ldots, 3$, left to right) encoding row i with bit value 2^j for entry (i, j); e.g. `1000` is E_{00} and `8421` the identity.

k	U	V	W	k	U	V	W	k	U	V	W
1	f07e	0366	2000	17	0808	1001	7e0b	33	0081	3067	7e7e
2	d05e	0360	3e1e	18	0e7e	b06d	b2b0	34	0809	1067	090b
3	0009	0067	0909	19	1010	5005	7eb0	35	585c	0144	7000
4	7e7e	00dd	8100	20	0909	10cd	0900	36	0080	3063	7e5e
5	0d7c	da6d	7a78	21	0001	7067	7e7c	37	0d0d	d0cd	7e78
6	0002	0760	0404	22	0004	0010	7e0f	38	0d5c	9a6d	7070
7	7a7e	01dd	0100	23	9090	0055	9000	39	007e	006d	1818
8	0e0e	d00d	3ab0	24	0d7e	0a6d	0808	40	000e	0060	3a1a
9	787c	0100	7a0b	25	707e	0066	2b0b	41	1e5e	d80d	3eb0
10	0070	d06d	3a38	26	0d4d	90cd	7e70	42	0090	306d	9090
11	787e	0166	0b0b	27	1c5c	984d	0e00	43	1090	3065	90b0
12	0f7e	ba6d	0200	28	1e7e	b86d	b0b0	44	d0d4	0054	7e0e
13	1818	1045	7e00	29	3e7e	08dd	8000	45	0202	dc0d	0400
14	0040	d08d	7ef0	30	2020	0500	4000	46	d05c	0364	0e0e
15	d090	005d	9010	31	0d09	00cd	0908	47	d07e	036d	1010
16	d0d0	0050	7e1e	32	0020	de6d	4040				

A.2 The programs

PLinOpt syntax. In L and R the inputs `i`$4i{+}j$ are a_{ij} (resp. b_{ij}) and output `o`k is the operand of product $k{+}1$; in P the inputs `i`k are the products m_{k+1} and output `o`$4i{+}j$ is c_{ij}. A line with t terms costs $t - 1$ XORs; copies (one term) are free.

L: 44 XORs

```
o2:=i12+i15;
o33:=i7+o2;
o18:=i0+i8;
o41:=i8+i11;
o42:=i11+o18;
o22:=i3+o42;
o29:=i1+i9;
o16:=i7+i15;
o14:=i2+o22;
o15:=i10+o14;
o19:=i4+o33;
o44:=i5+i13;
o43:=i14+o15;
o12:=o18+o16;
o32:=i11+i12;
o30:=i6+o19;
o36:=i14+o30;
```

```
o25:=i10+o36;
t18:=o22+o43;
o34:=o12+t18;
o8:=o29+o34;
o10:=i13+o8;
o6:=o44+o8;
o3:=i6+o6;
o24:=i7+o10;
o0:=i3+o24;
o46:=i1+o0;
o28:=i2+o3;
o40:=o29+o28;
o26:=o44+o40;
o1:=o29+o0;
o27:=i9+o40;
o17:=i0+o27;
o11:=i4+o17;
o23:=i5+o11;
o4:=o44+o11;
o37:=i9+o4;
o45:=i13+o1;
t38:=i12+o36;
o9:=o4+t38;
o7:=o17+o9;
t41:=o3+o24;
o39:=o7+t41;
o38:=o17+t41;
o5:=i13;
o13:=i10;
o20:=i12;
o21:=i14;
o31:=i9;
o35:=i11;
```

R: 49 XORs

```
o29:=i4+i6;
o15:=i8+i10;
o39:=i9+i10;
o16:=i0+i12;
o43:=i14+o15;
o22:=i12+o43;
o14:=i15+o22;
o3:=i11+o14;
o30:=i8+o3;
o28:=i7+o3;
o6:=i4+o3;
o19:=i0+o30;
o25:=i3+o19;
o36:=i2+o25;
o13:=i10+o36;
o7:=i11+o13;
o9:=o39+o7;
o40:=i7+o7;
o44:=i6+o40;
t19:=i8+o43;
o34:=i4+t19;
o12:=o16+t19;
t22:=o15+o14;
o38:=o39+t22;
t24:=i5+o38;
o46:=i4+t24;
o1:=t22+o46;
o5:=i6+o1;
o23:=i7+t24;
o45:=i14+o1;
o0:=i13+o45;
o10:=i5+o0;
o24:=i4+o10;
o2:=i12+o24;
o33:=o16+o24;
o32:=i1+o33;
o42:=i13+o32;
o41:=i15+o42;
o17:=i3+o41;
o20:=i2+o32;
o27:=i7+o17;
o35:=i14+o32;
o11:=i5+o27;
o37:=i1+o11;
o4:=i2+o37;
o31:=i6+o4;
t46:=o42+o27;
o26:=o12+t46;
o18:=o40+t46;
o8:=i4;
o21:=i8;
```

P: 74 XORs

```
y41:=i34+i12;
y40:=i45+y41;
y39:=i37+y40;
y38:=i26+i12+y39;
y34:=i21+i8+i16;
y33:=i43+y41+y34;
y31:=i15+y33;
y30:=i1+i39;
y29:=i35+y31+y30;
y28:=i32+y40+y29;
y27:=i20+y28;
y25:=i4+y27+i9;
y24:=i36+y25;
y22:=i25+y39+y24;
y21:=i13+y22;
y20:=i40+i18+y21;
y19:=i7+y20;
y16:=i10+i24;
y15:=i17+y19;
y14:=i27+y15;
y13:=i42+y14;
y12:=i41+y13;
y11:=i14+y12;
y10:=i5+i8+i39+y27;
y9:=i33+y16;
y8:=i31+y30+i9+y21;
y6:=i46+i38;
y5:=i23+i38;
y4:=i2+y9;
y3:=i30+y5+y4;
y2:=i22+y11;
y1:=i19+y3;
o0:=y6+y2;
o1:=i0+i24+y14;
o2:=i29+i18+y8;
o3:=i28+y19+y2+i3;
o4:=i6+y5+y1+i3;
o5:=i11+y38+y16+y15;
o6:=i44+y38+y25+y20+y10;
o7:=y38+y19+y1;
o8:=y33+y11+y6;
o9:=y29+y13;
o10:=y31+y8;
o11:=y22+y12;
o12:=y34+y4;
o13:=y28+y9;
o14:=i16+y10;
o15:=y24+y3;
```